

Competitive Programming

Section 52: Game Theory

Competitive Programming Course

2026-07-06

Table of contents

1	Winning and losing states	3
1.1	Normal-play impartial games	3
1.2	Subtraction game DP	4
1.3	Nim	4
1.4	Grundy numbers	6
1.5	DAG games	6
1.6	How to recognize game theory	8
1.7	Common mistakes	8
2	Exercises	9

1 Winning and losing states

Game theory in competitive programming is not about guessing what a clever opponent will do. Both players are assumed optimal. Your job is to classify states.

For normal-play impartial games:

- normal play means the player who cannot move loses;
- impartial means both players have the same legal moves from the same state;
- finite means the game eventually ends, or at least the state graph we solve is acyclic.

The central definition is small but powerful:

```
A state is winning  if it has at least one move to a losing state.  
A state is losing   if every legal move goes to a winning state.
```

That is the whole engine. Nim, subtraction games, and DAG games are different ways of evaluating this definition efficiently.

1.1 Normal-play impartial games

Start with a tiny game. There is one pile of stones. A move removes 1 or 2 stones. The player who takes the last stone wins because the next player has no move.

Let W mean winning and L mean losing.

```
stones: 0  1  2  3  4  5  6  
state:  L  W  W  L  W  W  L
```

Why?

- 0 is losing: no legal move.
- 1 is winning: move to 0.
- 2 is winning: move to 0.
- 3 is losing: moves go to 2 and 1, both winning.
- 4 is winning: move to 3.

The table is just the recursive definition evaluated from small states to large states.

The recursive definition is:

```
lose[state] = all next states are winning  
win[state]  = at least one next state is losing
```

Terminal states with no moves are losing.

1.2 Subtraction game DP

If there are i stones and allowed moves are in set S , then:

```
win[i] = exists x in S such that i >= x and win[i-x] == false
```

Example with $S = \{1, 3, 4\}$:

i	0	1	2	3	4	5	6	7
moves	-	0	1	2,0	3,1,0	...		
win?	L	W	L	W	W	W	W	L

At $i = 2$, the only legal move is to 1, which is winning, so 2 is losing. At $i = 3$, you can remove 3 and move to 0, so 3 is winning.

Minimal Python implementation:

```
def subtraction_winner(n, moves):
    win = [False] * (n + 1)
    for i in range(1, n + 1):
        for x in moves:
            if i >= x and not win[i - x]:
                win[i] = True
                break
    return win[n]
```

Minimal C++ implementation:

```
vector<int> win(n + 1, 0);
for (int i = 1; i <= n; i++) {
    for (int x : moves) {
        if (i >= x && !win[i - x]) {
            win[i] = 1;
            break;
        }
    }
}
```

The complexity is $O(n * |S|)$. This is enough when n is moderate. If n is huge, look for periodicity, but do not assume periodicity without proof.

1.3 Nim

For Nim piles, the invariant is xor:

```
x = pile_1 xor pile_2 xor ... xor pile_n
```

If $x == 0$, the position is losing. If $x != 0$, choose a pile whose highest set bit matches the highest set bit of x , and reduce it to:

```
new_pile = old_pile xor x
```

This makes the total xor zero. The opponent must then move away from xor zero, and you can restore it.

1.3.1 Worked Nim move

Suppose the piles are:

```
10, 12, 5
```

Binary:

```
10 = 1010
12 = 1100
 5 = 0101
xor= 0011
```

The xor is 3, so the position is winning. The highest set bit of 3 is the 2 bit. Pick a pile with that bit set; 10 = 1010 has it. Reduce it to:

```
10 xor 3 = 9
```

New piles:

```
9, 12, 5
```

Check:

```
 9 = 1001
12 = 1100
 5 = 0101
xor= 0000
```

Now the opponent faces a zero-xor position.

1.3.2 Why zero xor is losing

If total xor is zero and a player changes one pile, at least one bit changes in that pile. Since no other pile changed, the total xor becomes nonzero. So every move from xor zero goes to xor nonzero.

If total xor is nonzero, the construction above always finds a pile to reduce so that the new xor is zero. Therefore every nonzero-xor position has a move to a losing state.

1.4 Grundy numbers

Win/lose is enough for one game. Grundy numbers are needed when combining independent games.

```
grundy[state] = mex({grundy[next]})
```

`mex` means minimum excluded nonnegative integer.

A state is losing iff `grundy[state] == 0`. For a sum of independent games, xor the Grundy numbers. This generalizes Nim: each pile is a game whose Grundy number is its size.

1.4.1 Mex examples

```
mex({})      = 0
mex({0})     = 1
mex({1})     = 0
mex({0,1})   = 2
mex({0,2,3}) = 1
```

For the subtraction game $S = \{1,3,4\}$, Grundy values start:

i	0	1	2	3	4	5	6	7
next	-	0	1	2,0	3,1,0	...		
grundy	0	1	0	1	2	3	2	0

The win/lose classification is recovered by checking whether the Grundy number is zero.

1.4.2 Combining games

If you have independent components, compute one Grundy value per component and xor them:

```
total = g1 xor g2 xor ... xor gk
```

The combined position is losing iff `total == 0`.

This is why Grundy numbers matter: they allow a complicated impartial game to behave like Nim piles.

1.5 DAG games

If the game graph is a DAG, process vertices in reverse topological order. Every outgoing neighbor has already been computed, so `mex` is available.

```
for v in reversed(topological_order):
    grundy[v] = mex(grundy[to] for to in graph[v])
```

If the graph has cycles, this simple method does not apply without additional game theory.

1.5.1 Worked DAG example

Edges:

```
1 -> 2, 3
2 -> 4
3 -> 4, 5
4 -> none
5 -> none
```

Diagram:

```
  1
 /  \
2     3
|    / \
4     5
```

Compute from sinks upward:

```
grundy[4] = mex({}) = 0
grundy[5] = mex({}) = 0
grundy[2] = mex({0}) = 1
grundy[3] = mex({0,0}) = 1
grundy[1] = mex({1,1}) = 0
```

So starting at 1 is losing, while starting at 2 or 3 is winning.

1.5.2 DAG implementation pattern

If the input promises that all edges go from lower index to higher index, the reverse order $n-1 \dots 0$ is already topological. Otherwise compute a topological order first.

Python mex helper:

```
def mex(values):
    seen = set(values)
    g = 0
    while g in seen:
        g += 1
    return g
```

C++ mex helper for small outdegree:

```
int mex_of(const vector<int>& vals) {
    unordered_set<int> seen(vals.begin(), vals.end());
    int g = 0;
    while (seen.count(g)) g++;
    return g;
}
```

For performance, if the outdegree is large and values are small, use a vector of timestamps instead of clearing a set repeatedly.

1.6 How to recognize game theory

Look for statements like:

- “two players alternate”;
- “the player unable to move loses”;
- “both play optimally”;
- “remove stones”, “move a token”, or “choose an outgoing edge”;
- “print WIN/LOSE”.

Then ask:

Is the game finite?

Is it impartial?

Can I express each state by one integer, a DP index, or a DAG vertex?

Do I only need win/lose, or do independent components require Grundy numbers?

1.7 Common mistakes

- Treating a state as winning because it has many moves. It needs one move to a losing state.
- Treating a state as losing because it has a move to a winning state. All moves must go to winning states.
- Computing DAG Grundy values in the wrong direction.
- Forgetting that $\text{mex}(\{1\}) = 0$.
- Applying Nim xor to games that are not Nim piles without proving their Grundy values.

2 Exercises

- A. Nim Winner: For each Nim position, print WIN or LOSE.
- B. Subtraction Game: Given total stones and allowed moves, print whether the first player wins.
- C. DAG Game: For each start vertex in a DAG, print whether it is winning. # Contest checklist

State the invariant, choose the model, prove the operation, then code the smallest reliable version.